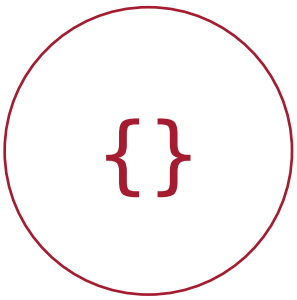
INSTITUTE OF RECURSIVE ADMINISTRATION

Faculty of Unnecessary Computer Science



The Typographical Filesystem

A Typst Implementation and a Doctoral Study of Creating Folders the Hard Way

The json2dir Research Collective

A satirical dissertation submitted for the fictional degree of

Doctor of Philosophical Directories

Supervisor: Professor Ada Mkdir
Examiners: Dr. Sybil Link and Professor Rex Cursion
Department of Braces, Permissions, and Related Drama

6 October 2026

FICTIONAL INSTITUTION - REAL SOURCE CODE

Declaration of Excessive Originality

This dissertation is a work of technical satire. Its institution, degree, supervisor, examiners, and ceremonial claims are fictional. Its implementation, interface contract, cited documentation, and reported test results are real.

The author declares that an ordinary task has been subjected to an extraordinary amount of typography. No committee was consulted before a typesetting language was asked to produce a shell program. Any resemblance to sensible engineering was discovered during testing and corrected only when it caused a bug.

The original json2dir implementation is credited to Alan Urmancheev. The implementation studied here is a new Typst port based on its documented conversion scheme and the locally inspected Rust source at revision `b2072fb5515a603024c123297b27dcaabfdf098a`. The original license is retained in the repository. The dissertation template is adapted from Ben Dierks's benplate 0.1.0 [1].

Generated file contents may include executable text, but the compiler creates that text as data; it does not run the scripts described by the input. The filesystem application program performs only the operations necessary to materialize the declared tree.

The json2dir Research Collective
Department of Having Already Gone Too Far
6 October 2026

Abstract

Directory creation has traditionally been entrusted to programming languages that admit an interest in operating systems. This dissertation challenges that disciplinary prejudice by implementing json2dir in Typst, a language whose principal professional qualification is arranging words on pages.

The proposed compiler accepts a JSON object, validates a four-constructor filesystem algebra, traverses its nodes with an explicit work stack, and emits a deterministic operation plan and a shell application program through Typst's experimental bundle export. Object values become directories, strings become UTF-8 files, and two-string arrays describe symbolic links and executable files. A thin host adapter transports standard input and executes the generated program; all conversion logic is expressed in Typst.

The implementation preserves byte-exact payloads by encoding UTF-8 bytes as fixed-width octal escapes. It validates the complete input before destination mutation, replaces existing file and link entries, merges directories, and refuses to replace a directory with a non-directory node. Validation yields 32 passing Typst unit assertions and 41 passing integration scenarios on the recorded host. These results establish tested behavior, not universal correctness, security against concurrent attackers, or performance superiority.

The principal conclusion is that a typesetting system can be made into a useful filesystem compiler, provided its author distinguishes compilation from execution and does not mistake a numbered theorem for a security proof. The secondary conclusion is that two braces can support a surprising amount of doctoral paperwork.

Keywords: Typst; JSON; filesystem synthesis; staged compilation; byte preservation; academic overcommitment.

Acknowledgments

I thank Professor Ada Mkdir for insisting that every contribution have a parent directory, Dr. Sybil Link for pointing somewhere else whenever a question became difficult, and Professor Rex Cursion for returning only after the base case had been funded.

I thank Alan Urmancheev for the original json2dir, the Typst maintainers for a language capable of participating in this incident, and Ben Dierks for a thesis template that gives even an unreasonable idea a respectable margin.

I am grateful to the test suite for rejecting the first version of this implementation. In particular, macOS's chmod utility declined to recognize a double dash, and ln attempted to place a symlink inside an existing directory. These interventions contributed more to the dissertation than any of its fictional reviewers.

This work received no grant funding. Its computational budget consisted of a local machine, an installed Typst compiler, and a refusal to choose a more appropriate language. The empty directory is thanked for its continued cooperation and its admirable publication record.

Contents

Declaration of Excessive Originality ii
Abstract iii
Acknowledgments iv
Contents v
1 Introduction: The Folder Deserves a Doctorate 1
1.1 A modest task with an immodest research programme 1
1.2 Research questions 1
1.3 Contributions 1
1.4 Scope and terminology 2
2 Background: Braces, Bytes, and Academic Authority 3
2.1 The original conversion scheme 3
2.2 Typst as an implementation language 3
2.3 Why generated shell is an appropriate intermediate target 3
2.4 Related work and template selection 4
3 A Semantics of Things in Other Things 5
3.1 The node algebra 5
3.2 The root and the name rule 5
3.3 Payload distinctions 5
3.4 Operational ordering and replacement 6
4 Architecture: A Compiler Wearing a Graduation Gown 7
4.1 Stages and artifacts 7
4.2 Separation of policy and effects 7
4.3 Three interfaces to one compilation 8
4.4 Failure boundaries 8
5 Implementation: The Typesetter Acquires Permissions 9
5.1 Validation as a total decision over parsed values 9
5.2 Iterative traversal 9
5.3 Quoting pathname operands 9
5.4 Encoding file contents as octal bytes 9
5.5 Directory and link details 10
6 Correctness: A Theorem Is Not a Backup 11
6.1 Assumptions and the strength of the claims 11
6.2 Proposition 1: Every successful plan contains accepted nodes 11
6.3 Proposition 2: Parent directories precede their descendants 11
6.4 Proposition 3: Encoded file payloads round-trip byte-for-byte 11
6.5 Proposition 4: Invalid trees do not mutate the destination 12
6.6 Corollary: A quiet empty object is a successful result 12
7 Evaluation: Seventy-Three Ways to Distrust a Folder 13
7.1 Method and recorded environment 13
7.2 Ordinary conversions and replacement scenarios 13
7.3 Adversarial payloads and invalid input 14
7.4 What the evaluation does not establish 14
8 Limitations: Reality Has Write Access 15
8.1 Experimental export and version boundaries 15
8.2 Concurrency and filesystem authority 15

8.3 Partial application and replacement policy 15
8.4 Input and metadata limits 15
8.5 Resource costs and integration boundaries 16
9 Discussion: The Political Economy of mkdir 17
9.1 What the unusual language choice reveals 17
9.2 Why the adapter does not invalidate the implementation 17
9.3 The value of boring tests in an unreasonable project 17
9.4 Satire as a documentation technique 18
9.5 Practical alternatives and future directions 18
10 Conclusion: QED, Then mkdir 19
10.1 Answers to the research questions 19
10.2 Deliverables and their relationship 19
10.3 Closing argument 19
A Reproduction Protocol 20
A.1 Requirements and commands 20
A.2 Source and license provenance 20
A.3 Interpreting generated records 20
B The Entire Typst Compiler 21
B.1 Library 21
B.2 Bundle entry point 23
C A Mock Oral Defense 24
C.1 Examiner: Why Typst? 24
C.2 Examiner: Is it really pure Typst? 24
C.3 Examiner: What is the most dangerous conclusion? 24
C.4 Examiner: What is the original contribution? 24
References 25

CHAPTER 1

Introduction: The Folder Deserves a Doctorate

1.1 A modest task with an immodest research programme

The original json2dir turns a JSON object into a directory tree. A name becomes a filesystem entry; its value decides what that entry contains. The interface is small enough to explain before coffee and useful enough to survive the coffee: feed JSON to standard input, obtain files in the current directory [2]. This dissertation asks what happens when that interface is entrusted to Typst.

The obvious objection is practical. Typst is a typesetting system. Its users normally ask it to lay out paragraphs, number equations, and persuade a table to remain within its margins. None of these activities implies a professional relationship with symbolic links. Yet Typst also has dictionaries, arrays, functions, loops, JSON loading, byte conversion, and an experimental facility for emitting arbitrary assets. The relevant question is therefore technical: which parts of filesystem synthesis can be expressed within that language, and which effects must cross a host boundary?

The project answers by treating json2dir as a compiler. Its source language is a restricted family of JSON values. Its intermediate representation is a sequence of validated operations. Its target language is an application program whose data operands are encoded by the compiler. This architecture makes the unusual language choice explicit instead of concealing an ordinary implementation behind a decorative PDF. The dissertation is absurd because it is a dissertation about this choice. The implementation must still work.

1.2 Research questions

The first question concerns expressiveness: can the complete conversion scheme be implemented in Typst without Rust, JavaScript, Python, a WebAssembly plugin, or a foreign implementation of the traversal? The answer depends on a precise boundary. Typst can parse and validate the input, choose operations, and generate the bytes of an application program. A host shell can execute that program. The shell adapter is part of deployment, and is disclosed as such.

The second question concerns fidelity: can generated programs preserve ordinary text, UTF-8, empty files, embedded NUL bytes, and all trailing newlines without turning user data into executable shell syntax? This is a stronger requirement than making the friendly example work. A directory archiver that quietly edits its payload is an editor with unusually aggressive output settings.

The third question concerns evidence: what does a passing local test suite actually demonstrate? It provides concrete observations for specified inputs and filesystem states. It does not prove resistance to concurrent attacks, establish asymptotic bounds for the Typst runtime, or give this fictional institute the power to award an accredited degree.

1.3 Contributions

The first contribution is a Typst implementation of the original four node kinds: directory, file, link, and script. The second is a staged design that completes validation before applying filesystem changes. The third is a byte-preserving shell encoder that keeps payload text out of the shell grammar. The fourth is an executable test suite with Typst unit assertions and host integration

checks. The fifth is an offline dissertation build using a pinned, locally adapted thesis template. The sixth, much less defensible contribution is the present manuscript.

These contributions differ in their status. The source files are executable artifacts. The test results are observations. The correctness arguments are informal reasoning under stated assumptions. The fictional academic apparatus is satire. Maintaining those distinctions is essential: the dissertation has already misused one category system by enrolling a directory in graduate school.

1.4 Scope and terminology

A *tree* means the abstract structure described by the accepted JSON values. A *plan* means the compiler's ordered sequence of operations. An *application* means executing those operations against a destination directory. A *script node* means a file that receives executable permission; its contents are never automatically executed as part of application.

The implementation targets local Unix-like environments, tested here on macOS. It is an experimental port, particularly because bundle export is explicitly experimental in Typst's documentation [3]. Windows behavior and hostile concurrent mutation are outside its tested contract. The goal is a small, reviewable implementation with an honest account of its boundary conditions. Readers seeking a universal deployment system are advised to follow a different symlink.

CHAPTER 2

Background: Braces, Bytes, and Academic Authority

2.1 The original conversion scheme

json2dir's useful idea is to let JSON's familiar structure carry a filesystem description. An object represents a directory; its keys provide entry names. A string represents a regular file's contents. A two-string array supplies a tag and a payload: the tag link selects a symbolic link, and script selects an executable file. The outermost value must be an object [2].

This scheme is deliberately narrower than JSON itself. JSON admits numbers, booleans, null values, strings, arrays, and objects [4]. json2dir accepts only the constructors that have an explicit filesystem interpretation. A number does not become its decimal textual representation, and a boolean does not become a directory named truth. Such conversions would be convenient inventions but would change the source language.

The locally inspected Rust implementation also defines important replacement behavior. It tries to remove each existing file or link entry before creating the requested one. Existing directories can be reused for directory nodes, allowing new children to be merged with unlisted entries. A directory is not recursively destroyed to make room for a file. The README's broad statement about deletion is therefore interpreted through the implementation's actual operations.

2.2 Typst as an implementation language

Typst's JSON loader converts JSON objects to dictionaries, arrays to arrays, strings to strings, booleans to booleans, and null to none. These mappings give the port a direct representation of its source data [5]. The implementation does not write a second JSON parser. Parsing is a standard-library boundary; validation and conversion are project code.

An explicit stack stores pending values and their paths. Each loop iteration removes one pending item. Directory values add their children; leaf values add one plan operation. This is ordinary algorithmic work expressed in a language normally encountered between a paragraph and its footnote. It requires neither a plugin nor the compiler author's willingness to call it a good idea.

The crucial output mechanism is Typst 0.15.1′s bundle target. Within a bundle, asset elements emit a chosen string or byte sequence to a specified output path. The compiler writes apply.sh and plan.json into an isolated staging directory. That facility supports the conversion artifact but does not itself provide an API for chmod, symbolic-link creation, or empty-directory creation [6]. The application stage addresses these remaining effects.

2.3 Why generated shell is an appropriate intermediate target

A generated shell program has three useful properties for this port. First, ordinary Unix hosts already provide a shell and the needed filesystem utilities. Second, the complete program can be inspected before execution. Third, Typst can generate it as text without accessing a foreign language evaluator.

The target also imposes obligations. Shell syntax is a programming language, not a neutral container. Filenames and payloads cannot be pasted into commands without encoding. A filename may contain a quote, a newline, a dollar sign, or text resembling command substitution. A file payload

may contain those features plus NUL bytes. The compiler must keep these values as operands, and the tests must exercise that claim with inputs designed to be inconvenient.

2.4 Related work and template selection

The original project is the direct behavioral reference. Typst's standard documentation supplies the language and export contracts. RFC 8259 supplies the JSON context. The printf specification describes the escape decoding relied upon by the generated program [7]. These are real sources; none is presented as endorsing this experiment.

The manuscript uses an adaptation of benplate 0.1.0 [1]. Its structure separates frontmatter, numbered chapters, appendices, and references. The local adaptation replaces external typography and helper-package dependencies with embedded fonts and native Typst rules. This makes the repository build offline once Typst is installed. The MIT license and provenance are included beside the template, because citations are not a substitute for license notices.

The fictional Institute of Recursive Administration contributes no technical authority. Its role is to provide a plausible place for a dissertation committee to ask whether an empty directory has sufficient novelty. The test suite is a more reliable examiner: it can fail the candidate without first scheduling lunch.

CHAPTER 3

A Semantics of Things in Other Things

3.1 The node algebra

Let a node be a directory, a textual file, a symbolic link, or an executable textual file. A directory maps acceptable entry names to further nodes. Files and scripts carry strings whose UTF-8 encoding becomes file content. Links carry strings that become link targets. In compact notation:

$$N ::= D(M) \mid F(s) \mid L(t) \mid X(s) \tag{1}$$

Here M is a finite mapping from names to nodes, s is a payload string, and t is a link-target string. The JSON encoding provides a concrete surface syntax for this algebra. The file and script constructors share a content representation but differ in the permission operation requested during application.

JSON form	Constructor	Application effect
Object	Directory	Create or reuse a directory
String	File	Write exactly the UTF-8 bytes
Two strings; tag link	Symbolic link	Create a link to the payload target
Two strings; tag script	Executable file	Write bytes, then add execute bits

Table 1: The complete accepted node vocabulary. A number is not a fifth constructor.

3.2 The root and the name rule

The root is an object describing entries inside an existing destination. The compiler never creates a named root entry. Therefore an empty object produces an empty operation sequence and leaves the destination untouched. This is a useful base case and an unusually accomplished doctoral candidate: it satisfies the specification by doing nothing.

An entry name must be non-empty, different from dot and dot-dot, contain no forward slash, and contain no NUL character. These constraints exclude explicit parent traversal and absolute paths. On the tested Unix host, a backslash is an ordinary filename character, so the validator allows it. A leading hyphen is also ordinary data; generated paths begin with dot-slash so that their names cannot be mistaken for utility options.

The rule is intentionally literal. The Rust path-component iterator can normalize some unusual names, such as a name ending in a slash. This port rejects every slash-bearing key, following the README’s stated one-component scheme without reproducing that normalization accident. Nested objects provide the supported way to describe nested paths. This difference is documented rather than smuggled into a compatibility claim.

3.3 Payload distinctions

A file string may contain a NUL character; its UTF-8 representation contains a zero byte, which is valid file content. A filename or symbolic-link target may not contain that character because the operating-system pathname interface cannot represent it. The validator checks names and link targets separately from file and script payloads. Conflating those domains would either reject valid file contents or emit impossible path operations.

A link target may be absolute, relative, dangling, or include parent traversal. It is the content of a link, not a child name. The converter does not recursively interpret it as another tree and does not require the target to exist. Whether a later consumer follows the link is a separate operation, carrying separate consequences. An empty target is left to host behavior, which is not claimed to be uniform across platforms.

3.4 Operational ordering and replacement

Within each directory, keys are sorted by the Typst runtime's string ordering. Children are pushed onto the pending stack in reverse order, so popping the stack produces sorted preorder traversal. Every directory operation precedes its descendants. This yields a deterministic plan for a parsed input tree, independent of its object insertion order. Duplicate JSON keys are outside the input contract: producers should use unique names, and the port relies on the standard parser's handling of duplicate names.

Application processes operations in plan order. Each operation first attempts to unlink an existing file or link at its path. Directory nodes then create a directory if necessary, leaving existing real directories available for merging. File and script nodes write their payload. Link nodes create the requested symbolic link and explicitly reject an existing directory. Script nodes add execute permission for user, group, and others after writing.

This is a sequential update semantics. It is not a transaction. Input validation completes before the first update, but an I/O error during application can leave earlier operations completed. A paper that described this as atomic would need more than a footnote; it would need a different implementation.

CHAPTER 4

Architecture: A Compiler Wearing a Graduation Gown

4.1 Stages and artifacts

The system has three stages. Transport copies the Typst sources into a private temporary workspace and saves standard input as input.json. Compilation loads that file, validates its tree, constructs a plan, and emits two assets. Application executes the generated asset within the caller's current directory. The temporary workspace is removed when the adapter exits.

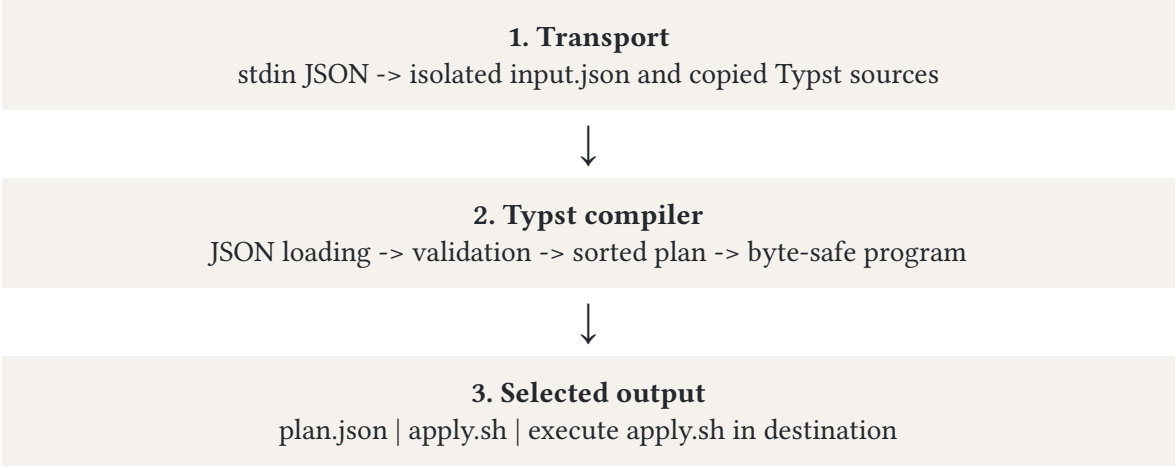


Figure 1: The system boundary. The arrows are architectural, not proof of inevitability.

The plan is a JSON document with a versioned schema identifier, an ordered nodes array, and summary statistics. Each node records kind, path, and payload. A directory's payload is null. The statistics count node kinds and encoded payload bytes, including link targets. They describe the input plan, not allocated disk space, filesystem metadata, or the amount of attention consumed by this thesis.

4.2 Separation of policy and effects

All conversion policy lives in src/lib.typ. The file specifies acceptable names, acceptable array tags, traversal ordering, node representation, quoting, octal encoding, operation emission, and statistics. src/main.typ connects JSON loading to that library and emits the assets. Neither source file imports a plugin or delegates conversion to another implementation.

The adapter in bin/json2dir handles arguments, temporary files, source transport, invocation, output selection, and cleanup. It does not examine JSON fields or decide which filesystem entries to create. In particular, there is no hidden jq, Python, JavaScript, or Rust interpreter deciding the tree's meaning. Generated shell commands carry the decisions already made by Typst.

This separation answers the phrase *pure Typst* at the level where it can be defended. The compiler implementation is pure Typst source, while actual host effects require a shell. A promise that Typst itself creates symbolic links or sets execute bits would be inaccurate. The project makes the boundary visible in its README, interface, tests, and dissertation rather than treating it as an embarrassing methodological appendix.

4.3 Three interfaces to one compilation

The default interface matches the original use: json2dir reads standard input and creates entries in the current directory. The plan option prints the validated plan without applying it. The emit option prints the generated shell program without applying it. Both inspection options leave the destination unchanged, although compilation still uses temporary staging files.

These interfaces enable review at different levels. The plan provides a semantic view of names, kinds, and payloads. The program provides the exact commands that would be executed. A user can retain the generated program and run it later in a chosen directory. Retention also transfers responsibility for selecting that later destination: the generated program deliberately contains relative paths rather than an embedded path to the original caller's directory.

4.4 Failure boundaries

Malformed JSON is rejected by Typst's JSON loader. Structurally invalid trees produce a structured validation error before source emission succeeds. The entry point promotes that error to a compiler diagnostic with a code and a path. The adapter's fail-fast execution means an unsuccessful compilation is never followed by application, even if some temporary output were present.

Filesystem failures occur during the final stage. The generated program uses fail-fast shell behavior so that a failed write, directory creation, link creation, or permission change terminates application. Its explicit unlink attempt is allowed to fail because an existing directory is a valid condition for directory reuse. This exception is purposeful; indiscriminately ignoring every error would make the system easier to publish and harder to use.

The isolation between staging and destination is also practical. Input can request entries named src, apply.sh, or plan.json without altering the compiler's own running copies, because those copies and artifacts live elsewhere. The resulting tree remains capable of replacing entries in the chosen destination, which is precisely the behavior the caller requested.

CHAPTER 5

Implementation: The Typesetter Acquires Permissions

5.1 Validation as a total decision over parsed values

The plan function returns either a successful operation sequence or a failure record containing a code, path, and explanation. It checks the root before initializing traversal. Each pending value is classified by its Typst type. Dictionaries introduce children; strings introduce files; arrays must contain exactly two strings and a recognized tag. Every other type fails explicitly.

The checks do not perform destination writes. This means an invalid late node cannot create earlier valid siblings. For example, a tree with a valid a entry and an invalid z entry is rejected before a appears on disk. That behavior is a deliberate improvement over the original interleaving of validation and effects, and it is exercised by an integration test.

Error paths provide context without changing the language's semantics. The path dot-slash-a-slash-b identifies a nested bad node; a parent path identifies a bad child name. The compiler's diagnostic also points into the source function that raised the error. This is less polished than a dedicated CLI error renderer, but avoids inventing a second validator merely to make failure look elegant.

5.2 Iterative traversal

The pending stack starts with the root item. Directory items append a directory operation unless they are the root, then add child items in reverse sorted order. The loop finishes when the stack is empty or returns immediately upon validation failure. The implementation therefore avoids recursion in its own traversal.

That design does not imply unlimited nesting. The JSON parser, Typst evaluation limits, memory allocator, host pathname limits, and filesystem remain relevant. An explicit stack removes one specific recursion mechanism; it does not repeal the laws of finite computers. This distinction is especially useful in a thesis, where the word unlimited is otherwise tempted to appear near a funding request.

5.3 Quoting pathname operands

The shell encoder surrounds an operand with single quotes. An embedded quote is represented by closing the quoted string, emitting a backslash-escaped quote, and reopening the quoted string. This preserves the operand while preventing dollar signs, backticks, and parentheses from becoming shell syntax. The paths are also prefixed with dot-slash, preventing option-like names from being passed as leading options to utilities.

The generated chmod invocation intentionally omits a double-dash separator. The locally tested macOS utility interprets that separator as an operand in this invocation. Its absence is safe here because the mode is fixed and the generated path starts with dot-slash. Other utility invocations use the separators supported by the tested host. Portability is established by observed host behavior rather than a sweeping claim that every utility on every Unix agrees.

5.4 Encoding file contents as octal bytes

Directly placing a payload in a shell string is insufficient because shell operands cannot hold embedded NUL bytes. Command substitution also removes trailing newlines. The compiler instead

converts the string to UTF-8 bytes and maps each byte to a fixed-width printf escape. For a byte b with $0 \le b \le 255$, its octal digits are:

$$d_2 = \left\lfloor \frac{b}{64} \right\rfloor, \quad d_1 = \left\lfloor \frac{b}{8} \right\rfloor \bmod 8, \quad d_0 = b \bmod 8 \tag{2}$$

The emitted representation is a backslash, zero, and these three digits. The generated printf percent-b command decodes the representation directly into the destination file. Payload data never passes through a shell variable or command substitution. Literal backslashes therefore remain literal, and trailing newlines remain part of the file. Empty payloads use an explicitly empty operand.

The representation uses five ASCII bytes per original byte before fixed command overhead. That overhead is a transparent cost. It buys a uniform path for all UTF-8 content, including zero bytes produced by escaped NUL characters in JSON. A future implementation could emit content assets and generate copy commands, but this dissertation studies the simpler self-contained application program.

5.5 Directory and link details

Existing directories require special care because the ln utility can interpret a directory operand as a place in which to create a link. The emitter explicitly rejects that condition before a link operation. Without the check, a successful command could create the wrong entry and report success. The integration suite caught precisely that bug during implementation.

Existing symlinks are unlinked before replacement. A directory node additionally checks that its path is not a surviving symlink. These checks reduce accidental traversal of pre-existing links in an ordinary local application. They do not provide descriptor-relative race protection. The security limitations follow from the host operations chosen, regardless of how attractively their names are typeset.

CHAPTER 6

Correctness: A Theorem Is Not a Backup

6.1 Assumptions and the strength of the claims

The following arguments concern the project algorithm and generated program. They assume successful JSON parsing, a finite input tree, the specified behavior of Typst's value operations, working host utilities, sufficient resources, and no concurrent modification of relevant destination paths. They are informal arguments, not machine-checked proofs. The numbered presentation is academic decoration with organizational value, not an escalation of evidential authority.

The target postcondition is also bounded. Requested directory entries have the requested kind and payload after successful application. Unlisted entries in reused directories remain. File permissions follow the host's creation behavior, with execute bits additionally enabled for script nodes. Ownership, timestamps, extended attributes, and directory metadata are not specified by the source language. A correct conversion should not acquire obligations the input cannot express.

6.2 Proposition 1: Every successful plan contains accepted nodes

Consider one iteration of the traversal. Its item is either a dictionary, string, tagged array, or another parsed value. A dictionary contributes only a directory operation and pending children whose names pass validation. A string contributes only a file operation. An array contributes only a link or script operation after its arity, element types, and tag have passed their checks. Every remaining case returns a failure instead of a successful plan.

Inductively, the accumulated nodes contain only accepted operations. Successful termination means every pending item has been classified without failure. The returned sequence therefore covers the accepted tree and contains no operation for an unsupported boolean, number, null value, or malformed array. This argument concerns the planner; the JSON loader's correctness remains a dependency.

6.3 Proposition 2: Parent directories precede their descendants

When a non-root directory item is removed from the stack, its operation is added before child items are introduced. No descendant can be removed from the stack before it has been introduced by an ancestor. Every ancestor directory operation therefore precedes the descendant's operation. The root needs no operation because application starts inside an existing destination directory.

Sorting and reversing the child insertion order additionally ensure that the next child removed from the stack has the first sorted name. Its descendants are introduced above remaining siblings, yielding depth-first preorder. This establishes determinism at the project level under a fixed parser result and runtime ordering, while avoiding a claim about arbitrary Unicode collation standards that the project neither implements nor requires.

6.4 Proposition 3: Encoded file payloads round-trip byte-for-byte

For any byte b, the digit calculation decomposes it into three base-eight digits. The digits satisfy $b = 64d_2 + 8d_1 + d_0$. Prefixing their fixed-width representation with the printf octal escape marker produces a code that decodes to b. Concatenation of those codes decodes to the original byte sequence, because each code has a fixed boundary and printf writes the decoded bytes in order.

The shell operand contains only ASCII backslash-zero-digit sequences and is single-quoted. The payload cannot introduce a printf cancellation escape or shell syntax because every original byte,

including a backslash, is encoded. The empty sequence receives an empty operand and produces an empty file. No trailing newline is synthesized or removed. The test suite confirms selected instances with byte comparisons, including UTF-8 and embedded NUL payloads.

6.5 Proposition 4: Invalid trees do not mutate the destination

Transport and compilation operate in temporary staging. Application is invoked only after compilation exits successfully. Structural validation fails by returning an error that the entry point promotes to a compiler failure. The adapter consequently exits without invoking the generated program. Invalid trees therefore cannot cause destination writes through this adapter, although temporary staging files were created and later removed.

This guarantee is about invalid input, not application failure. An accepted tree can still encounter permission errors, missing resources, or incompatible existing entries. Those failures terminate the program after any earlier effects. A destination snapshot or transaction protocol would be required for rollback. The distinction prevents the very common scientific error of proving a property of phase one and advertising it as a property of the entire universe.

6.6 Corollary: A quiet empty object is a successful result

An empty root introduces no child items and produces no operations. Its generated program contains initialization but no filesystem update commands. Application therefore succeeds without changing destination entries under the stated assumptions. This corollary is offered to the committee as the theoretical justification for taking an afternoon off.

CHAPTER 7

Evaluation: Seventy-Three Ways to Distrust a Folder

7.1 Method and recorded environment

Evaluation uses executable checks instead of invented speedups. The repository's test runner compiles Typst assertions, generates fixtures in Typst, applies them in isolated temporary directories, and verifies host-visible outcomes. After a successful run, a Typst report emitter records the passing counts, compiler version, platform, and UTC timestamp. This chapter reads that record directly, so its headline numbers are connected to an artifact rather than manually transcribed into prose.

The recorded compiler is `typst 0.15.1 (unknown commit)`. The recorded platform is `Darwin 27.0.0 arm64`. The verification record was produced at `2026-10-06T18:59:00Z`. The result contains 32 passing unit assertions and 41 passing integration scenarios, with zero recorded failures. A scenario can contain multiple assertions; adding the counts describes suite entries, not a rigorous count of independent experimental observations.

Evidence layer	Passed	What it exercises
Typst unit suite	32	Validation, traversal, quoting, byte encoding, statistics
Host integration suite	41	Files, directories, links, modes, errors, CLI inspection
Performance benchmark	Not run	No comparative performance claim
Formal verification	Not performed	Informal arguments only

Table 2: Recorded evidence and deliberately absent evidence. Blank ambition is not a measurement.

7.2 Ordinary conversions and replacement scenarios

The upstream example exercises all four node kinds. Its greeting is checked, nested file contents are compared byte-for-byte, its empty subdirectory is observed, its symbolic-link target is read, and its script has executable permission and produces the expected output when the test explicitly runs it. The converter itself never runs that payload script.

Replacement scenarios begin with existing files, links, and directories. They verify file replacement, file-to-directory conversion, directory reuse, and preservation of unlisted children. Existing file and directory symlinks are replaced without modifying their targets. Attempts to turn a real directory into a file, script, or link fail while preserving its existing child. These checks distinguish intentional merging from accidental recursive deletion.

An empty-root scenario confirms that unrelated destination entries remain unchanged. Inspection scenarios confirm that plan and emit modes do not apply the tree. The emitted program is subsequently executed in an isolated destination to verify that it carries the necessary behavior. The plan output is compared with direct Typst compilation of the same example, checking the adapter's input transport and output selection.

7.3 Adversarial payloads and invalid input

The hostile fixture uses a filename containing a single quote, a newline, dollar signs, backticks, and text resembling command substitution. Its payload contains literal shell-looking text, backslashes, multiple trailing newlines, and an embedded zero byte. The test compares the resulting file with bytes independently emitted by Typst's asset facility. It also confirms that marker files named by the apparent command substitutions were not created.

Additional entries exercise an option-like filename, a non-ASCII filename and payload, a literal backslash in a name, and a link target beginning with a hyphen and containing a newline. A separate script fixture checks both embedded NUL content and executable permission. The tests use byte comparisons where shell string comparison would lose information, particularly for trailing newlines and NUL bytes.

Invalid fixtures cover malformed JSON, non-object roots, unsupported leaf values, array lengths, array element types, unrecognized tags, invalid names, and a NUL inside a link target. Every invalid scenario starts with a sentinel file and checks that it survives as the only destination entry. A late invalid sibling specifically exercises validation-before-application behavior.

7.4 What the evaluation does not establish

The suite does not reproduce the original Rust project's full region coverage, fault-injected permission failures, or every platform-specific error. It does not test concurrent attackers, Windows, network filesystems, pathological input sizes, or every allowed filename. Its claims apply to the tested scenarios and recorded host. A passing count should motivate confidence in those behaviors, not erase the boundary of the experiment.

No runtime benchmark against Rust is reported. The compiler performs parsing, sorting, value traversal, byte encoding, staging, and program generation before starting filesystem effects. Those stages have costs, and byte encoding expands payload size. Their existence supports a qualitative cost discussion, but cannot support a numerical speed comparison. The fictional committee requested a graph with an upward arrow; the evaluation declined on methodological grounds.

CHAPTER 8

Limitations: Reality Has Write Access

8.1 Experimental export and version boundaries

Bundle export is explicitly experimental and enabled through a feature flag [3]. This repository was tested with Typst 0.15.1. An installation that lacks the bundle target or asset support cannot run this compiler without changing its output mechanism. Future compiler releases may change experimental behavior. The build therefore states its tested version rather than treating the latest release as a timeless mathematical constant.

The dissertation's normal PDF build does not depend on bundle export, but the implementation and verification reports do. The local template does not need network access or downloaded helper packages. That distinction keeps document build reproducibility separate from runtime portability. A beautiful PDF is not evidence that the command-line converter executed successfully.

8.2 Concurrency and filesystem authority

The generated program uses path-based utilities. Between an unlink, directory check, creation, and write, another process may change a path. The project does not use descriptor-relative operations or provide protection against time-of-check to time-of-use races. Its deliberate removal of pre-existing symlinks helps ordinary replacement behavior but cannot establish confinement in a directory that an adversary can mutate concurrently.

The input also controls symbolic-link targets. A target can point outside the destination, and another application may later follow that link. This is a feature of the format, not an implicit promise that every resulting link stays inside the tree. Users choose the destination and authority under which the generated program runs. The repository demonstrates a local conversion tool, not a privilege boundary or a service for applying untrusted trees as root.

8.3 Partial application and replacement policy

Validation rejects unsupported input before destination mutation. Accepted input can nevertheless fail halfway through application. A directory may be unwritable, a path may be too long, a filesystem may be full, or a requested link may be unrepresentable on the host. Earlier successful operations remain. The program has no rollback journal and does not promise all-or-nothing application.

Directory merging is similarly specific. Unlisted children remain in an existing directory. If a consumer expects an exact mirror in which absent names are deleted, this converter will not supply one. Conversely, replacing a real directory with a file or link is rejected rather than recursively deleting its contents. These choices follow the inspected implementation's broad behavior and keep deletion authority limited to existing file and link entries.

8.4 Input and metadata limits

The format describes UTF-8 text, including characters that encode to zero bytes. It does not provide an arbitrary binary-file encoding. It does not specify owner, group, ACLs, extended attributes, special devices, hard links, timestamps, or chosen permission masks. Script nodes add execute permission to the normal file creation mode; they do not request a particular complete mode such as 0755. The host umask remains relevant.

Strict key validation rejects slash-bearing names even when the Rust path iterator might normalize them to one component. Backslashes remain ordinary Unix name characters. Duplicate keys are outside the documented producer contract. Filename length limits and case sensitivity remain filesystem properties. The operating system can refuse a name that is structurally valid under the source language; compile-time validation cannot predict all host state.

8.5 Resource costs and integration boundaries

The adapter reads the entire input into a staging file, and Typst loads the parsed tree. The planner stores its pending work and final operation sequence. The byte encoder constructs an escaped representation, and the emitter retains the generated program text. Large payloads therefore incur storage and memory costs beyond their final on-disk size. The implementation does not stream input or apply operations incrementally during parsing.

The explicit work stack avoids the project traversal's function-call recursion, but it cannot remove parser limits or runtime resource limits. No maximum-size or maximum-depth performance claim is made. Native Windows application is not implemented. Linux may work with the selected utilities, but the recorded verification is macOS-only. The defense committee's honorary penguin does not count as a Linux test machine.

CHAPTER 9

Discussion: The Political Economy of mkdir

9.1 What the unusual language choice reveals

Implementing json2dir in Typst makes a familiar distinction unusually visible: describing effects is different from performing them. A planner can be expressed in a language with no conventional filesystem mutation API. The planner's output is a program, which a host executes later. This observation is ordinary compiler design viewed through an unreasonable lens, and that is the experiment's main educational value.

The staged architecture improves review. A caller can inspect the complete plan or application program before running it. The same architecture improves input failure behavior by finishing validation before application. Neither benefit depends on the degree certificate. Both depend on how the implementation connects its stages and prevents failed compilation from reaching execution.

The language choice also concentrates pressure on representation boundaries. The author cannot casually use a native write-file function and assume its string handling is correct. Instead, the payload must cross JSON decoding, Typst UTF-8 encoding, generated shell syntax, printf decoding, and filesystem writing. Each boundary provides a place for corruption or injection. The octal encoder and byte comparison tests are responses to that concrete chain, not ceremonial security vocabulary added after the cover page.

9.2 Why the adapter does not invalidate the implementation

Every deployed implementation relies on a runtime and an operating system. The relevant question is which project logic is delegated and whether the delegation is disclosed. Here the adapter is a transport and invocation layer. The compiler determines the complete operation sequence, path operands, content operands, and permission operations in Typst. A host shell executes that generated sequence.

Calling the compiler a pure Typst implementation is accurate under that stated boundary. Calling the entire operating-system interaction Typst-native would be inaccurate. The repository's source layout provides evidence for the distinction: two Typst compiler files, Typst fixture and unit files, a shell adapter, and a shell integration runner. No source from the original Rust implementation is needed at runtime. The manuscript includes the Typst compiler in full so that the claim can be reviewed without accepting its abstract.

9.3 The value of boring tests in an unreasonable project

The strongest moments in development were mundane failures. The first empty payload test exposed an empty-array join returning none rather than an empty string. A script permission check exposed a utility-option portability problem. A directory-to-link check exposed ln's directory destination behavior. Each failure concerned an ordinary edge case with a concrete operational consequence.

This pattern argues for tests aligned with the contract. Byte comparison checks content fidelity. Link-target checks inspect links themselves. Directory merge checks observe unlisted entries. Invalid-input checks observe absence of destination effects. A test that merely reproduced the emitter's source string would be weaker evidence for filesystem behavior, although small unit assertions remain useful for localized encoding and validation rules.

9.4 Satire as a documentation technique

The fictional degree gives the manuscript room to exaggerate the importance of a small tool while keeping its technical claims proportionate. An empty object can be called a distinguished scholar, provided the next paragraph correctly states that it emits no operations. An examiner can be named Sybil Link, provided an actual symlink target is checked with an actual host utility. The joke depends on the reader recognizing where exaggeration ends.

That separation also limits fabricated authority. The manuscript cites real documentation, credits the original author and template, and reports observed test counts. It does not invent measured benchmark results, endorsement by a real university, or a formal proof certificate. The companion diploma labels its institution and degree fictional. The research collective receives a title; the reader receives source code and a reproducible build.

9.5 Practical alternatives and future directions

A direct asset backend could emit file payloads separately and generate a smaller application program that copies them into place. A transaction-aware backend could stage a complete replacement tree for selected use cases, although merge semantics and host atomicity would need an explicit design. A hardened native executor could consume the plan using descriptor-relative filesystem operations. That executor would change the pure-Typst boundary and should be named honestly.

These ideas are future work, not features already present. Their existence does not justify postponing the simpler implementation's tests. The current system has a clear interface, real artifacts, recorded behavior, and limitations visible before deployment. In the Institute of Recursive Administration, this is known as the minimum publishable directory.

CHAPTER 10

Conclusion: QED, Then mkdir

10.1 Answers to the research questions

The complete json2dir conversion logic can be implemented in Typst. The compiler loads JSON through the standard library, validates its accepted constructors, builds a sorted preorder plan with an explicit work stack, quotes path operands, encodes payload bytes, and emits an application program and inspection plan. No Rust, Python, JavaScript, or plugin supplies the conversion algorithm.

Actual directory creation, symbolic-link creation, and permission changes cross an explicit host boundary. A small shell adapter stages the input, invokes the compiler, selects output, and executes the generated program when requested. This answers the expressiveness question without attributing unsupported host APIs to Typst. The implementation is unusual enough without requiring an inaccurate description of what it does.

Byte-preserving conversion is achieved by encoding each UTF-8 byte into a fixed-width octal printf escape. This removes payload text from shell syntax, preserves embedded zero bytes in file contents, and retains trailing newlines. The tested fixtures include hostile-looking names and payloads, UTF-8, empty values, symbolic links, and executable files. Their passing results provide concrete support for the implemented encoding path.

Evaluation establishes 32 passing unit assertions and 41 passing integration scenarios on the recorded host. It also identifies what has not been established: comparative performance, formal verification, hostile concurrency safety, broad platform portability, or unlimited input size. Those omissions are part of the result's meaning, not material to be hidden until the examiner has signed.

10.2 Deliverables and their relationship

The repository contains the Typst compiler sources, a compatible stdin-based command interface, inspection modes, fixtures, unit tests, integration tests, a machine-readable verification record, and English documentation. It also contains this dissertation's Typst source, its local template with provenance and license, its rendered PDF, and a companion fictional diploma.

The source is the primary evidence for implementation claims. The suite is the primary evidence for tested behavior. The verification record connects the document's counts to the latest successful local run. The PDF makes the material readable and searchable. The diploma has exactly the authority stated on its face: none outside the fictional institute. These artifacts serve different purposes, and no amount of typography makes them interchangeable.

10.3 Closing argument

The result is a working compiler with a visible effect boundary, accompanied by a dissertation built with the same language. Typst has successfully produced both the program and the paperwork needed to explain its questionable career move.

The Rust implementation remains next door. This port fulfills its own brief: recreate the conversion in Typst, make the result executable, and document it with more academic ceremony than a folder could reasonably demand.

The candidate therefore submits the sources, the tests, and the manuscript. The committee may now deliberate. Its deliberation directory already exists, its unlisted minutes have been preserved, and its honorary symlink points to the nearest coffee machine. The base case has been reached.

CHAPTER A

Reproduction Protocol

A.1 Requirements and commands

Install Typst 0.15.1 and provide a Unix-like host with sh, mktemp, cp, cat, rm, mkdir, ln, chmod, cmp, find, wc, tr, and make. The implementation was verified on the platform recorded in the evaluation chapter. No Typst package download is required by the local template or compiler.
From the repository root, run:

```
make all
make demo
```

The first command runs the suite, updates the verification record, and compiles both PDFs. The second creates the supplied example tree under build/demo. For conversion in a different destination:

```
mkdir -p /tmp/my-json2dir-output
cd /tmp/my-json2dir-output
/path/to/json2dir-thesis/bin/json2dir < /path/to/tree.json
```

The generic paths above are examples to replace with local paths. Inspection does not apply the tree:

```
bin/json2dir --plan < examples/tree.json
bin/json2dir --emit < examples/tree.json
```

The compiler can also be called directly without the adapter:

```
typst compile --features bundle --format bundle --root . \
  src/main.typ build/bundle
```

Its default input is the repository example. A different file inside the selected project root can be provided with the tree input option. The bundle contains apply.sh and plan.json. Executing apply.sh separately applies its tree to the directory where that generated program is run.

A.2 Source and license provenance

The original json2dir was read from the neighboring project. Its inspected revision is `b2072fb5515a603024c123297b27dcaabfdf098a`. Its license text is retained at the repository root; its source language is not required to build the port. The MIT license for benplate is retained under templates. The adapted template has no remote imports, and uses fonts embedded in the Typst compiler.

A.3 Interpreting generated records

verification/results.json records a successful test run. It is not an input acceptance certificate or a formal proof. Re-running make test refreshes the record, which may change the compiler version, platform, time, and test counts. Rebuilding the dissertation reads that refreshed record. The stable document date refers to the project edition, not a claim that all future tests ran on the same day.

CHAPTER B

The Entire Typst Compiler

The following listing is read directly from the implementation source at build time. It is not a manually maintained transcription. Its presence turns the dissertation into a rather literal example of literate overcommitment.

B.1 Library

```
// SPDX-License-Identifier: ISC
// A filesystem compiler written entirely in Typst. No plugins or foreign code.

#let failure(code, path, message) = (
  ok: false, nodes: (), error: (code: code, path: path, message: message),
)

#let check-name(name, parent: ".") = {
  if name == "" or name == "." or name == ".." {
    return failure("invalid-name", parent, "Expected a non-special, non-empty
filename")
  }
  if name.contains("/") {
    return failure("invalid-name", parent, "Names must have exactly one path
component")
  }
  if name.contains("\u{0}") {
    return failure("invalid-name", parent, "A filename cannot contain a NUL byte")
  }
  (ok: true,)
}

// An explicit work stack avoids recursive function calls for nested directories.
// A successful result is a deterministic, sorted, preorder operation sequence.
#let plan(tree) = {
  if type(tree) != dictionary {
    return failure("invalid-root", ".", "Expected the JSON document to be an
object")
  }
  let pending = ((path: ".", value: tree, root: true),)
  let nodes = ()
  while pending.len() > 0 {
    let item = pending.pop()
    let value = item.value
    let kind = type(value)
    if kind == dictionary {
      if not item.root {
        nodes.push((kind: "directory", path: item.path, payload: none))
      }
      for name in value.keys().sorted().rev() {
        let checked = check-name(name, parent: item.path)
        if not checked.ok { return checked }
        pending.push((path: item.path + "/" + name, value: value.at(name), root:
false))
```

```
      }
    } else if kind == str {
      nodes.push((kind: "file", path: item.path, payload: value))
    } else if kind == array {
      if value.len() != 2 or type(value.at(0)) != str or type(value.at(1)) != str {
        return failure("invalid-array", item.path,
          "Expected an array of two strings: [kind, payload]")
      }
      let tag = value.at(0)
      let payload = value.at(1)
      if tag != "link" and tag != "script" {
        return failure("invalid-kind", item.path,
          "Expected array kind to be link or script")
      }
      if tag == "link" and payload.contains("\u{0}") {
        return failure("invalid-target", item.path,
          "A symbolic-link target cannot contain a NUL byte")
      }
      nodes.push((kind: tag, path: item.path, payload: payload))
    } else {
      return failure("invalid-node", item.path,
        "Expected an object, string, or tagged array")
    }
  }
  (ok: true, nodes: nodes, error: none)
}

#let require-plan(tree) = {
  let result = plan(tree)
  if not result.ok {
    panic(result.error.code + " at " + repr(result.error.path)
      + ": " + result.error.message)
  }
  result.nodes
}

// POSIX shell single-quote encoding. Data is never interpreted as shell source.
#let shell-quote(value) = "'" + value.replace("'", "'\\''") + "'"

// printf %b accepts \0ooo. Encoding every UTF-8 byte also preserves embedded
// NULs in file contents, literal backslashes, and all trailing newlines.
#let octal(value) = {
  let data = bytes(value)
  let digits = "01234567"
  let encoded = ()
  for index in range(data.len()) {
    let n = data.at(index)
    encoded.push("\\0" + digits.at(calc.floor(n / 64))
      + digits.at(calc.rem(calc.floor(n / 8), 8)) + digits.at(calc.rem(n, 8)))
  }
  if encoded.len() == 0 { "" } else { encoded.join() }
}

#let emit-shell(nodes) = {
  let lines = ("#!/bin/sh",
    "# Generated by json2dir in Typst. Run inside the destination directory.",
    "set -eu")
  for node in nodes {
    let path = shell-quote(node.path)
```

```
    // Like upstream remove_file: remove files and links, leave directories.
    lines.push("rm -f -- " + path + " 2>/dev/null || :")
    if node.kind == "directory" {
      lines.push("if [ ! -d " + path + " ]; then mkdir -- " + path + "; fi")
      lines.push("[ ! -L " + path + " ] || { printf '%s\\n' "
        + "'Refusing a directory symlink' >&2; exit 1; }")
    } else if node.kind == "link" {
      lines.push("[ ! -d " + path + " ] || { printf '%s\\n' "
        + "'Cannot replace a directory with a symlink' >&2; exit 1; }")
      lines.push("ln -s -- " + shell-quote(node.payload) + " " + path)
    } else {
      lines.push("printf '%b' " + shell-quote(octal(node.payload)) + " > " + path)
      if node.kind == "script" { lines.push("chmod a+x " + path) }
    }
  }
  lines.join("\n") + "\n"
}

#let statistics(nodes) = (
  total: nodes.len(),
  directories: nodes.filter(n => n.kind == "directory").len(),
  files: nodes.filter(n => n.kind == "file").len(),
  links: nodes.filter(n => n.kind == "link").len(),
  scripts: nodes.filter(n => n.kind == "script").len(),
  payload-bytes: nodes.filter(n => n.payload != none)
    .fold(0, (sum, n) => sum + bytes(n.payload).len()),
)
```

B.2 Bundle entry point

```
// SPDX-License-Identifier: ISC
#import "lib.typ": require-plan, emit-shell, statistics
#let tree = json(sys.inputs.at("tree", default: "/examples/tree.json"))
#let nodes = require-plan(tree)
#asset("apply.sh", emit-shell(nodes))
#asset("plan.json", json.encode((schema: "json2dir-plan/v1",
  nodes: nodes, statistics: statistics(nodes))))
```

CHAPTER C

A Mock Oral Defense

C.1 Examiner: Why Typst?

Candidate: Because the brief required Typst, and its scripting and experimental bundle facilities are sufficient to compile the conversion program. The choice also exposes the boundary between planning and filesystem effects unusually clearly. I have not claimed that it is the fastest or most conventional choice.

Examiner: Could you have stopped after writing mkdir?

Candidate: Certainly. But the four-constructor format also needs file contents, links, and executable bits. More seriously, a real port needs the replacement and error cases. Less seriously, stopping there would have left insufficient material for the Faculty of Unnecessary Computer Science.

C.2 Examiner: Is it really pure Typst?

Candidate: The parser binding, validator, traversal, representation, quoting, byte encoder, statistics, and emitter are Typst. A shell adapter performs transport and invokes the emitted program. That boundary is documented. Typst itself does not expose native symlink or chmod operations in this implementation.

Examiner: Have you hidden Python somewhere?

Candidate: No runtime or test conversion logic uses Python. The source and fixture generation are Typst, and host integration checks are shell. The dissertation does not need a foreign-code plugin.

C.3 Examiner: What is the most dangerous conclusion?

Candidate: That passing tests make a path-based program safe against concurrent attackers. They do not. The program does not provide descriptor-relative race protection, rollback, or untrusted-root confinement. Input validation is useful, but it is not a substitute for a threat model and a different host backend.

Examiner: Why no graph showing Typst defeating Rust?

Candidate: Because no comparative benchmark was run. A fictional graph would be an entertaining picture and an invalid experimental result. The present thesis already contains enough fiction in its institutional stationery.

C.4 Examiner: What is the original contribution?

Candidate: A working Typst compiler for the conversion, an explicit execution boundary, byte-preserving generated payload handling, and tests for real filesystem outcomes. The intellectual contribution is modest. The typography has been allowed to remain ambitious.

Examiner: Your degree is approved, subject to one correction.

Candidate: Which correction?

Examiner: Please place the diploma next to the dissertation.

Candidate: Already done.

References

[1] B. Dierks, “benplate 0.1.0: A Flexible Thesis Template.” [Online]. Available: https://typst.app/universe/package/benplate

[2] A. Urmancheev, “json2dir: Directory Archives, Made Human-Readable.” [Online]. Available: https://github.com/alurm/json2dir

[3] Typst GmbH, “Typst Documentation: Bundle Export.” [Online]. Available: https://typst.app/docs/reference/bundle/

[4] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format.” [Online]. Available: https://www.rfc-editor.org/rfc/rfc8259

[5] Typst GmbH, “Typst Documentation: JSON.” [Online]. Available: https://typst.app/docs/reference/data-loading/json/

[6] Typst GmbH, “Typst Documentation: Asset.” [Online]. Available: https://typst.app/docs/reference/model/asset/

[7] The Open Group, “POSIX.1-2024: printf.” [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/utilities/printf.html